StudentPay AI — Algorithm Description

Step-by-step computational logic for every major feature in the submitted prototype

All algorithms below are derived from the submitted JavaScript implementation. They describe what the prototype actually computes, not a proposed future backend.

A. Registration

1. Read username, email and password from the form.
2. Check username against the pattern of 3–30 letters, numbers or underscores.
3. Validate the email using the prototype's basic email pattern.
4. Require at least 8 password characters, including at least one letter and one number.
5. Normalize the username to lowercase for the storage key.
6. Reject the registration if the username or email already exists.
7. Generate a random salt.
8. Compute SHA-256 over salt + password.
9. Store username, email, salt and hash in the browser database.
10. Write a registration event to the local audit log.

B. Login and Local Lockout

11. Read username and password.
12. Normalize username to lowercase.
13. Load failed-attempt timestamps for that username.
14. Keep only attempts from the previous 15 minutes.
15. If five or more attempts remain, reject the login and show the approximate remaining lockout time.
16. If an account exists, hash the entered password with that account's stored salt.
17. Compare the generated hash with the stored hash.
18. On failure, append the current time to the failed-attempt list and record a login_failed event.
19. On success, clear failed attempts, record login_success, set the active user and open the application.

C. Transfer Comparison

20. Read the amount, sending currency, recipient currency and mid-market rate.
21. Read every provider option and its rate, fee and fee currency.
22. Validate the amount, rate and fee.
23. If the fee is charged in the sending currency, subtract the fee before conversion.
24. If the fee is charged in the recipient currency, convert the full amount and subtract the recipient-currency fee.
25. Sort all valid providers by estimated recipient amount, descending.
26. Mark the first row as best.
27. For every provider, calculate the difference from the best result.

28. If a positive mid-market rate is available, calculate ideal recipient amount = amount × mid-market rate.
29. Calculate hidden cost = ideal recipient amount − estimated received amount.
30. Calculate hidden-cost percentage = hidden cost / ideal recipient amount × 100.
31. Render the comparison table and enable saving.

D. Affordability Calculator

32. Read monthly available income/funds.
33. Read Rent, Food, Transportation, Tuition and Other expenses.
34. Reject invalid or negative values.
35. Sum all expenses.
36. Calculate remaining = income − total expenses.
37. Calculate percentage spent = total expenses / income × 100 when income is positive.
38. If remaining < 0, show an error state.
39. Else if remaining is below 10% of income, show a warning.
40. Otherwise show a healthy-cushion state.
41. Save the budget values for the current user.

E. Survival Planner

42. Read current balance, days until income, expected income, daily essential spending and bills due before income.
43. Validate the balance, day range, income, daily spending and bills.
44. Calculate available after bills = balance + expected income − bills.
45. Calculate essential forecast = daily spending × days.
46. Calculate estimated remaining = available after bills − essential forecast.
47. Calculate daily spending ceiling = available after bills / days, bounded at zero.
48. If estimated remaining is negative, report a potential shortfall.
49. If remaining is positive but less than 10% of the essential forecast (with a minimum threshold of 1), report a tight budget.
50. Otherwise report that the estimate covers the period with a buffer.
51. Save the latest inputs and render suggested next steps.

F. Transaction Tracker

52. Read description, type, category, amount, date and currency.
53. Reject blank descriptions or non-positive amounts.
54. Create a transaction record with a timestamp-based ID.
55. Insert it at the start of the user's transaction list.
56. Record an audit event and persist the database.
57. Recalculate recorded income, expenses and net total for display.
58. For CSV export, create a header row and quote/escape each field before generating a downloadable CSV blob.

G. Bills / Tuition Guardian

59. Read bill name, amount, due date, category and currency.
60. Validate required fields and positive amount.
61. Store the bill as unpaid.
62. Sort bills by due date for display.
63. Classify each bill as Paid, Overdue, Due today or Upcoming.
64. Flag unpaid bills due within seven days with a warning state.
65. Allow the user to toggle paid/unpaid or delete the bill.
66. Update the dashboard summary after changes.

H. Savings Goals

67. Read goal name, target amount, saved amount and number of months.
68. Require a name, positive target, non-negative saved amount and at least one month.
69. Store the goal for the current user.
70. Calculate progress percentage = saved / target × 100, capped at 100.
71. Calculate monthly required saving = max(0, target − saved) / months.
72. Show a goal-reached state when saved ≥ target.
73. Allow additional contributions and deletion.

I. Dashboard Score and Tips

74. Start with a baseline score of 30.
75. If a budget exists, derive a score from the remaining budget; cap the budget contribution at 80.
76. Add 10 if a saved transfer comparison exists.
77. Add 10 if at least one savings goal exists.
78. Cap the final score at 100.
79. Assign a state: healthy at 70+, warning at 45–69, error below 45.
80. Generate tips from missing budget/comparison/goal/transaction data and overdue or unpaid bills.
81. Display the result as a prototype financial-health indicator, not a regulated credit or financial score.

J. Scam Shield

82. Display six warning statements.
83. Count the checked warning statements.
84. Zero flags → low/no red flags message.
85. One or two flags → caution message.
86. Three or more flags → high scam-risk message advising the user not to send money.
87. This is a heuristic checklist and does not perform independent provider verification.

K. AI Assistant

88. Build a context string from the current user's budget, goals, transaction totals, bills, survival estimate and latest transfer comparison.
89. Create a system instruction telling the assistant to provide concise educational guidance and never claim to execute payments.

90. If the provided Claude sample runtime is available, send recent chat context plus the system prompt and user question.
91. Stream/update the assistant message as text arrives.
92. If the AI runtime is unavailable, call the local deterministic advisor.
93. The local advisor uses keyword matching to answer questions about hidden costs, transfers, budget, survival, bills, transactions and savings.
94. Render the final response in the chat panel.

L. Persistence and Activity Log

95. Maintain a single JavaScript DB object in memory.
96. Serialize the DB object as JSON.
97. Store it under the browser localStorage key sp_v1.
98. On application load, attempt to restore sp_v1.
99. On relevant actions, append an audit event and retain the latest 300 global audit records.
100. When displaying activity, filter the audit list to the currently logged-in prototype user.

Algorithmic Assumptions

- Currency values are treated as numbers in the currency units the user enters; the prototype does not perform currency conversion from live data.
- Transfer comparison assumes the entered rate and fee are correct.
- Budget and survival outputs are arithmetic estimates based only on user-entered values.
- The dashboard score is a product heuristic, not a statistically validated financial-health model.
- The scam shield is a warning checklist, not an identity, license or fraud-detection service.